An Embedded Systems Cookbook So You Don't Get Cooked

Stepan Pressl

What Is an Embedded System?

An embedded system is a specialized computer system—a combination of a computer processor, computer memory, and input/output peripheral devices—that has a dedicated function within a larger mechanical or electronic system. It is embedded as part of a complete device, often including electrical or electronic hardware and mechanical parts. Because an embedded system typically controls physical operations of the machine that it is embedded within, it often has real-time computing constraints.

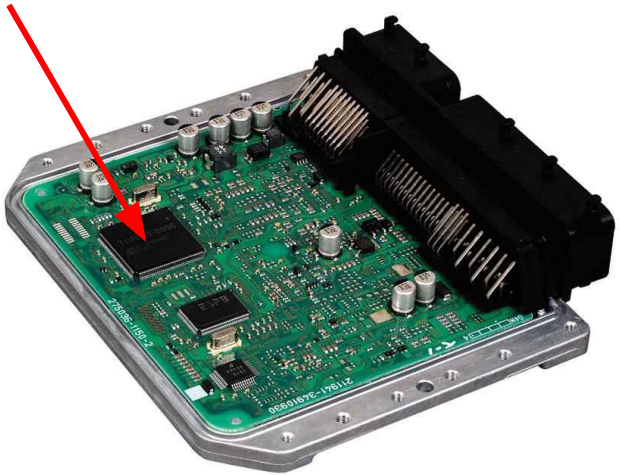
Embedded systems control many devices in common use. In 2009, it was estimated that 98% of all microprocessors manufactured were used in embedded systems.

Source: Wikipedia (Embedded system)

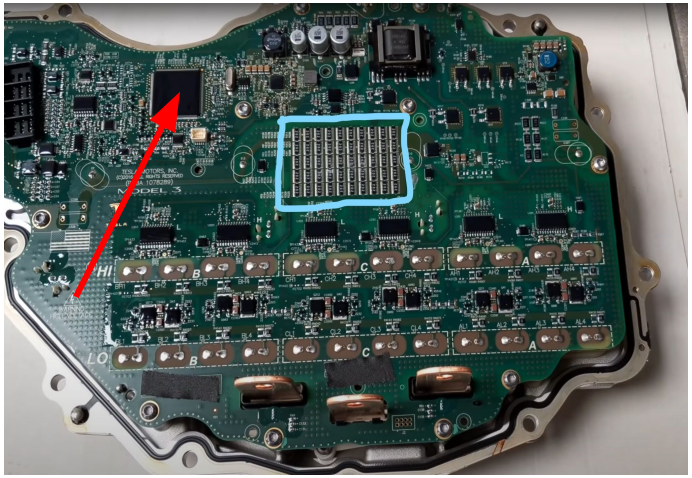
Terminology and Basic Knowledge Base

- Integrated circuit (abbreviated as IC)
- Microcontroller (abbreviated as MCU)
  - A programmable computer with a dedicated memory and I/O (input and output) peripherals
- Firmware: a piece of software developed for embedded systems
- Firmware development somewhere between electrical engineering and computer science
- Used everywhere: consumer electronics, computers, transportation, cars, medical equipment, military and defense, space.
- MCUs typically programmed in C/C++/Rust

Typical MCU Application Examples



Car ECU: gathers data from various sensors (shaft positions, temperatures, gas concentration) to control fuel injection and ignition.



Electric Motor controllers (here you can see one from a Tesla vehicle)

Smart Kettle Controller :-)

Outline

1. Why are MCUs used?
2. MCU Peripherals and what are they used for?
3. Which MCU should you choose for your own project?
4. MCU programming paradigms (polling, interrupt and scheduling with workers)
5. Overview of Scheduling and Introduction to Real-Time Operating Systems
6. NuttX RTOS Example with a STM32 Nucleo board

Why are MCUs used?

Why are MCUs used?

- A simple comparison:

STM32F767

- 13.44 USD (Mouser, if you buy 10)
- 1 ARM Cortex M7 core @ 216 MHz

AMD Ryzen 5 5500

- 84 USD (Amazon)
- 6 x86 cores @ 4.2 GHz

*If you do the math, the STM32F767 would cost $4200/216 \cdot 6 = 1568$ USD if it had the same computational power as Ryzen.

* comparing computational power of different architectures using clock speed is not a good metric, it is a very simplified example

Why are MCUs used?

- Compared to PC processors, much easier to program
- Used for control purposes (acquire data from sensors or different I/Os, react to events and control something - valves, motors, lights, etc.)
- Offer easily programmable peripherals commonly used with sensors and industrial equipment
- Much simpler to setup
- Less power hungry

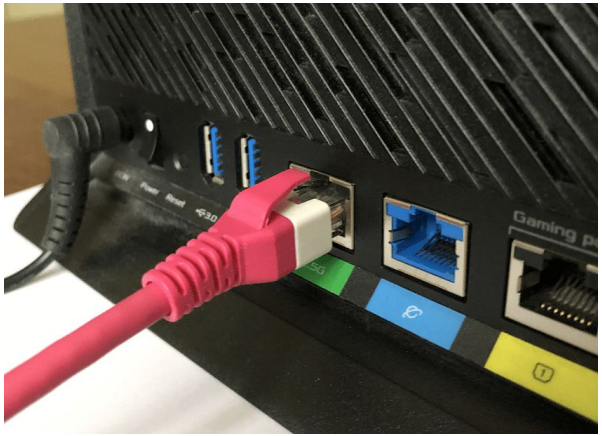
Typical MCU Peripherals

Typical MCU Peripherals

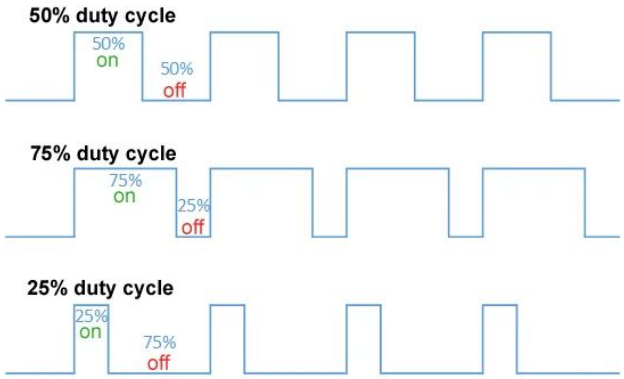
- GPIOs (General Purpose Inputs and Outputs) - logical 0/1
  - used for reading digital inputs or turning on switches (relays, transistors, pins of other ICs)
- Analog-to-Digital converter (ADC)
  - converts analog voltage signal to a digital number (from which you can compute the true analog value in code)
- UART (serial line)
  - Receive/Transmit pair, the simplest communication peripheral, every MCU has one
  - Pro tip: always include a debugging UART for hardware/firmware bringup
  - different physical layer variations: RS232, RS422, RS485
    - Higher voltage levels or differential pair signalling enable long distance communication in noisy environments.
- I2C, SPI
  - used primarily for communication with sensors and ICs (mainly configuration or data acquisition)
- Interrupts
  - technically not a peripheral, but allows to interrupt the code execution when an event happens

Typical MCU Peripherals

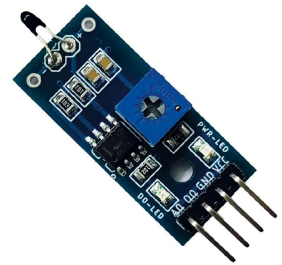
- Timers
  - enables time measurement or programmable timing events (i.e. every 20ms do something)
- PWM
  - a peripheral that enables the generation of a PWM signal with a programmable duty and period (used primarily for motor control, light intensity control)
- CAN bus
  - a communication bus for networks without a master node
  - commonly used in distributed networks with a lot of nodes (such as cars)
- USB
- Ethernet (IEEE 802.3)
  - you probably know this
  - allows you to implement protocols which are commonly used in every day life (such as UDP/IP, TCP/IP, HTTP, etc.)



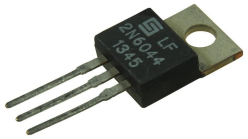
Ethernet RJ45 Connector



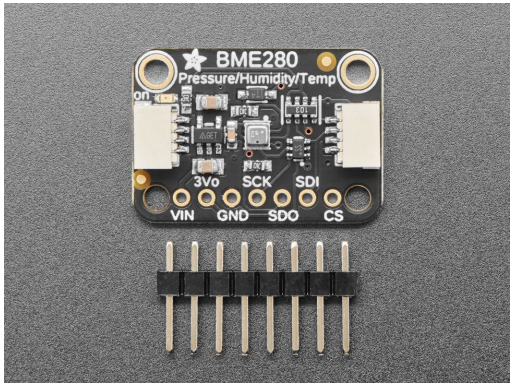
PWM Waveforms



Temperature sensor with an analog output (use ADC)



GPIO Controlled Components



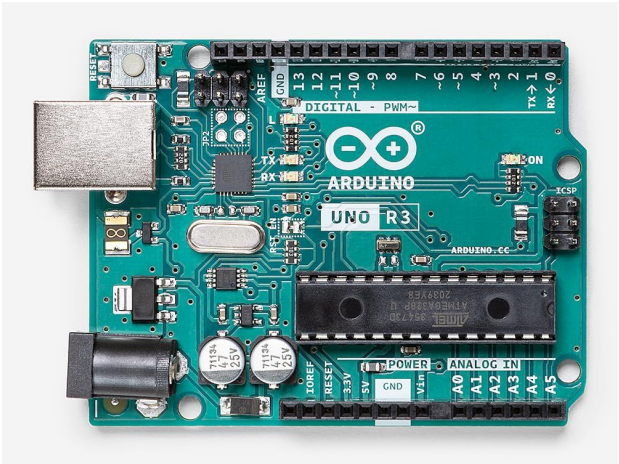
Humidity sensor, communication over I2C or SPI)

What MCU Should You Choose for Your Project?

What MCU Should You Choose for Your Project

Arduino

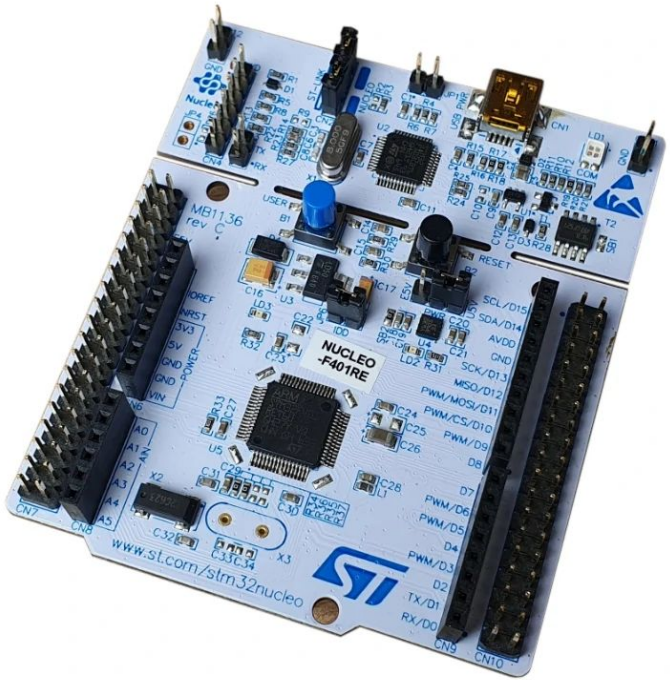
- Based on an AVR ATmega microcontroller
- Objectively outdated
    - low computational power,
    - low clock speed,
    - limited number of peripherals (no CAN, Ethernet, USB)
- Very popular, due to a large community and large code base



What MCU Should You Choose for Your Project

STM32 MCU Series from STMicroelectronics

- Based on ARM Cortex M cores
- Very popular with large community and large code base
- Aimed for consumer and industrial applications
- The more expensive ones offer all peripherals you can think of
- Bang for a buck compared to ATMega



What MCU Should You Choose for Your Project

ESP32 and Nordic Semiconductor MCUs

- known for vast support of wireless communication protocols (WiFi, Bluetooth, Zigbee, etc.)
- Commonly used in IoT devices and wearables
- ESP32s are a huge bang for a buck

What MCU Should You Choose for Your Project

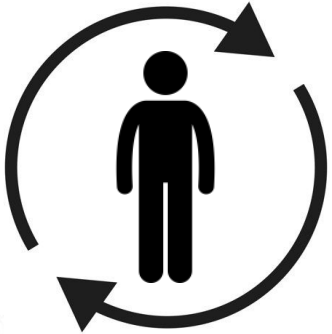
- Arduino/AVR ATMega for quick prototypes
  - Easy to learn, but I would not use it in my products. Ditch it if you want to grow in the embedded world.
- ESP32 or Nordic Semiconductor chips for wireless communication
- STM32 is a standard, spanning from consumer to industrial applications
- Pro tip: always use something that fits your requirements with an established community and available toolchains (programs which compile the C/C++ code into the executable code)
- There are a lot of other MCUs for specific applications. But let's keep it simple.

MCU Programming Paradigms

(i.e how you write your application code for your MCU)

MCU Programming Paradigms (Polling)

- Based on an infinite while loop that executes all necessary tasks.
- You continuously check every task. If the task needs attention, you perform action.
- You cannot write blocking code inside the loop.

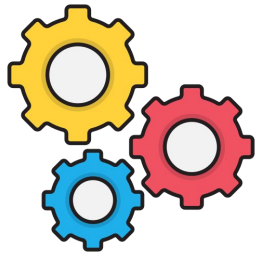
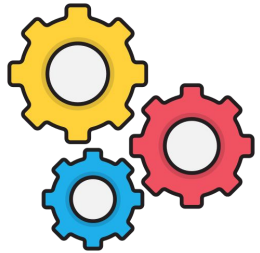
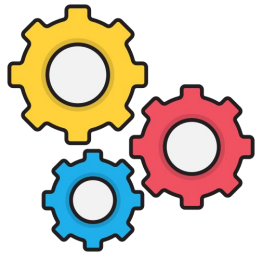


MCU Programming Paradigms (Bare-Metal Interrupt)

You don't do anything. You only register interrupts. If an interrupt fires up, you perform action.

MCU Programming Paradigms (Workers)

- You have multiple independent workers that are normally in a waiting (blocking) state.
- When an event occurs and a task needs attention, the worker performs the action, and then returns to sleep (blocks) again.


MCU Programming Paradigms Summarized

- "Bare-metal" way
  - Combination of polling and interrupt approaches
  - Difficult if dependencies between the actions exist - you need to keep state
  - Polling is resource intensive (why ask if you're asleep?)
- "Workers" way
  - Useful for more concurrent tasks
  - More convenient to think of independent workers

Typical MCU Application Example

You need to program a microcontroller to control the speed of a DC motor. The target speed is always read from RS485 (serial line) and is immediately applied.

You also need a control loop responsible for keeping the target motor speed (due to external impacts). Another requirement is to detect motor faults - temperature exceeded and overcurrent.

Q/A 1: What MCU peripherals do you need to use?

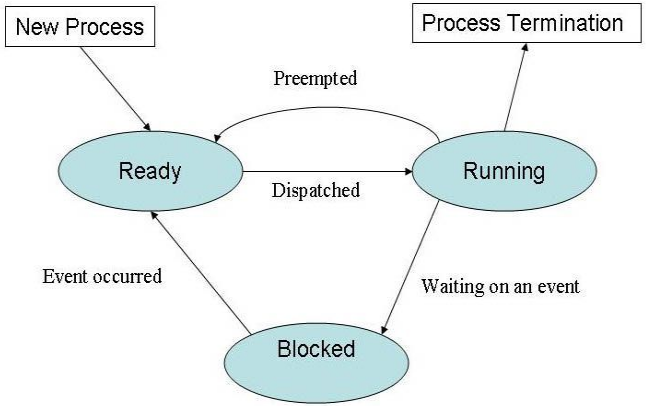
Q/A 2: How would the bare-metal polling loop look like?

Q/A 3: How would it look like when you used workers?

Overview of Scheduling and Introduction to Real-Time Operating Systems

Overview of Scheduling

- MCU cannot run multiple workers simultaneously
- Scheduler: keeps track of all tasks
- If a task needs to wake up, it signals and the scheduler transfers execution (context switch).
- Task priority: a higher priority task can interrupt a running task (preemption) with lower priority
- Normally, a scheduler quickly switches from one task to another (every ~1 ms-10 ms) (round-robin), if every task is running



The Task State Machine

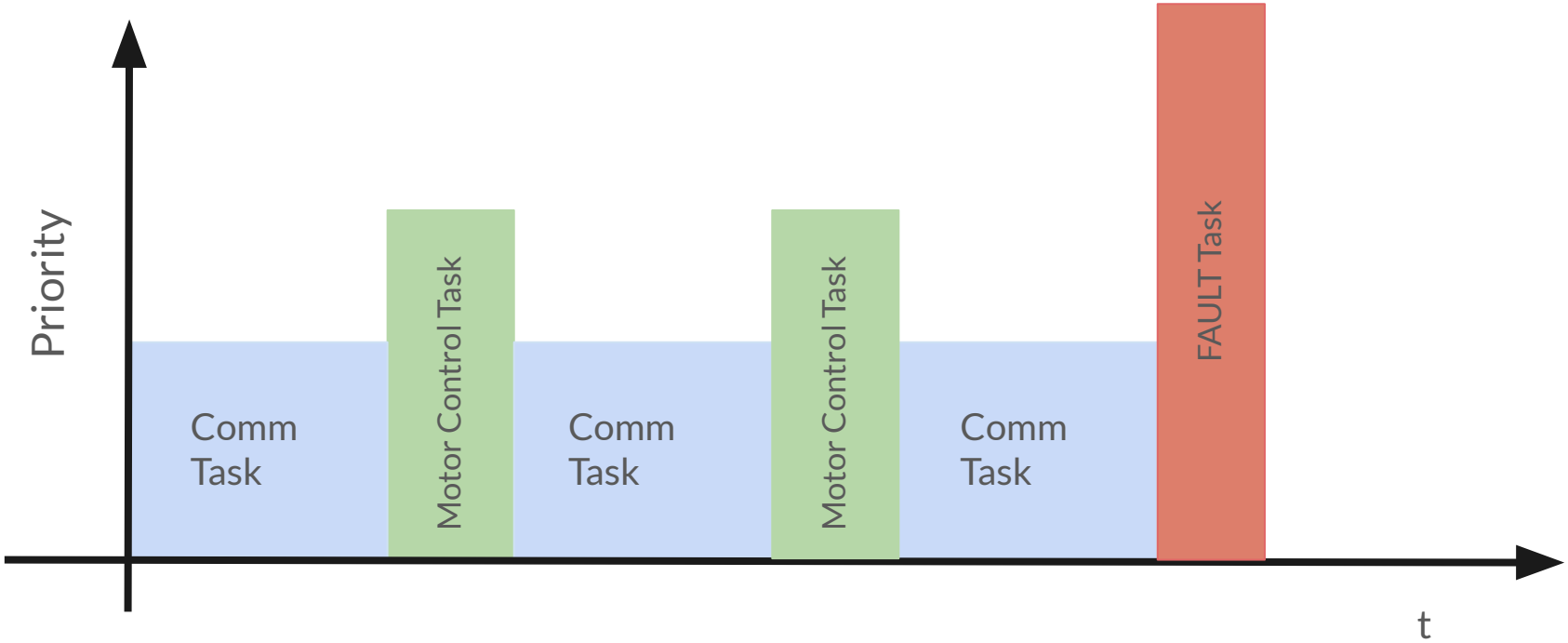
https://scriptcrunch.com/linux-process-management-tutorial/

Real-Time Operating Systems

- Implements the scheduler and ways to
  - create and manage tasks, the tasks run in threads
  - a way of signalling a task needs attention (semaphores, queues, specific function calls)
- Most of them also implement unified drivers (UART, USB, Ethernet, CAN, etc.)
  - driver is an unified software interface that provides access to hardware peripherals
- Implementation in C/C++/Rust
- What does Real-Time mean?
  - A guaranteed time upper bound on context switch latency for high-priority tasks.
- All together = kernel

Q/A: Have you heard of Linux? Have you also heard of the Linux Kernel?

Back to the DC Motor Example



Real-Time Operating Systems (cont'd)

- **Pros**: application written in RTOS is architecturally independent; easy to write very complex applications with asynchronous tasks
- **Cons**: kernel has a resource overhead; not suitable for simple applications
- NuttX, Zephyr, VxWorks, ThreadX, FreeRTOS, Linux RT
- Nuttx
  - POSIX compliance (follows the same coding principles as Linux)
  - "Everything is a file", even a device driver
  - Support for STM32, ESP32, AVR ATMega and many others

Usage of NuttX (using C programming language)

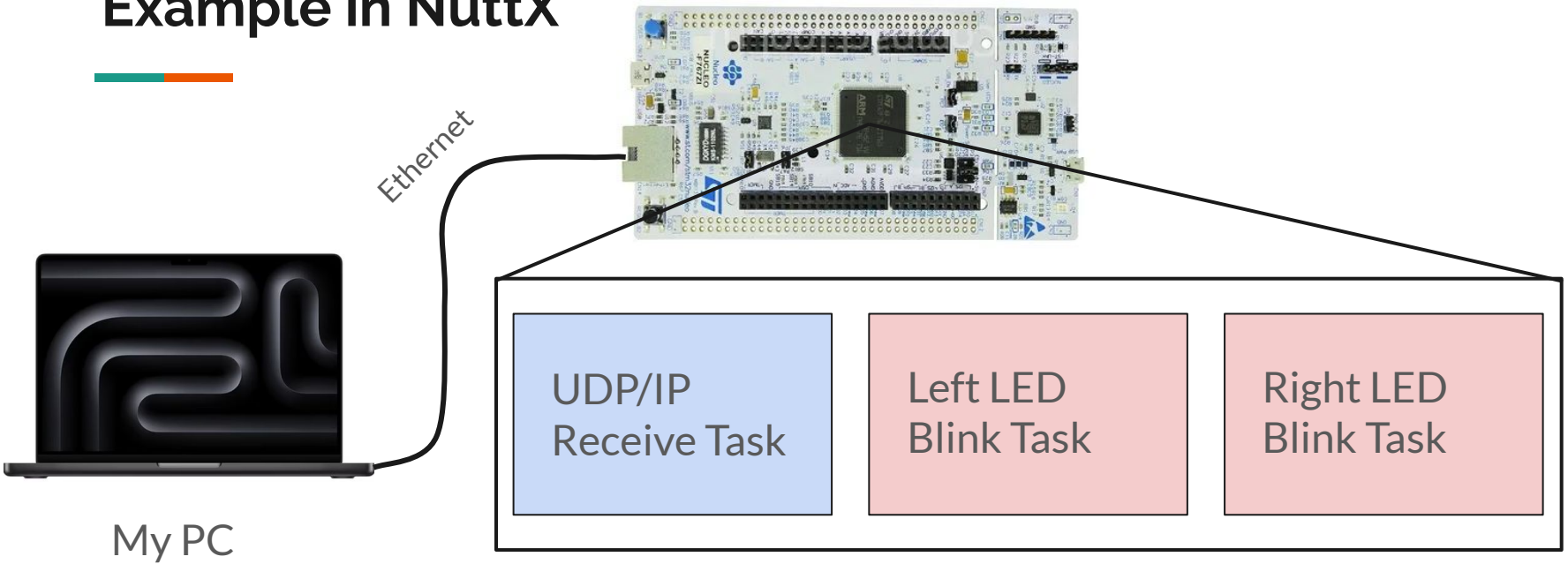
```
void *blink(void *data) {
     int fd = open("/dev/gpio1", O_RDWR);
     if (fd < 0) {
          return NULL;
     }

     int on = false;
     while (true) {
          ioctl(fd, GPIOC_WRITE, on);
          on = !on;
          usleep(1000 * 100);
     }
     close(fd);
     return NULL;
}
```

GPIO Blinking Loop

```
pthread_t thrd1, thrd2, thrd3;
pthread_create(&thrd1, NULL, blink, NULL);
pthread_create(&thrd2, NULL, blink, NULL);
pthread_create(&thrd3, NULL, blink, NULL);
```

Create 3 tasks with blinking loops

Example in NuttX



I will create 3 tasks: one for receiving commands (w,s,e,d) from my computer, and two LED blinking tasks. If w or s is pressed, the Left LED blinking period is modified via a global variable. The same for Right LED using e or d.

Conclusion

- RTOS is an OVERKILL for such application. Just wanted to show you what is possible.
- Honestly, there is a steep learning curve in embedded, but I've always liked it!
- Use whatever you want, as long as you have fun experimenting!
- I hope you can apply this new engineering knowledge to grow!

Thank you! And let's keep in touch!

github.com/zdebanos

Štěpán Pressl

Presentation Link (PDF)